

ONLINE TICKET BOOKING SYSTEM

Java Programming Project Report

Course Code	CSE2006
Student	Prince Kumar
Registration Number	25BAI11079
University	VIT Bhopal University
Project Type	Java / Object-Oriented Programming

A console-based Java application for user authentication, show selection, seat booking, ticket generation and booking management.

Abstract

The Online Ticket Booking System is a Java-based console application designed to simplify the process of browsing shows, selecting available seats, creating bookings and generating tickets. The project separates customer and administrative operations and uses multiple classes to demonstrate object-oriented programming, modular design, collections, validation and exception handling.

The application is intentionally console-based so that the implementation stays focused on Java programming and OOP concepts while still representing a realistic ticket-booking workflow. The design diagrams in this report describe the users, process flow, sequence of operations and main classes used by the application.

Introduction

Booking tickets manually can involve repeated checking of show details, seat availability, customer information and payment or booking records. A simple computerized system can organize these activities and reduce common input and calculation errors.

The proposed system provides a straightforward workflow in which a user logs in, views available shows, selects seats, confirms a booking and receives ticket information. Administrative functions are used for managing show-related information.

Problem Statement

An organization that offers ticket-based shows needs a simple way to maintain show information and allow users to reserve available seats. The system should prevent invalid seat selection, calculate the booking amount correctly and provide a clear ticket after confirmation.

Objectives

- Implement a user login flow with basic validation.
- Display available shows and their relevant details.
- Allow users to select available seats.
- Create and manage booking records.

- Calculate the total booking amount.
- Generate ticket details after successful booking.
- Provide administrative functions for show management.
- Demonstrate classes, objects, inheritance, collections and exception handling.

Requirements Analysis

Functional Requirements

ID	Requirement	Description
FR1	User Login	Authenticate the user before booking operations.
FR2	Show Display	Display available shows with relevant details.
FR3	Seat Selection	Accept seat choices and check availability.
FR4	Booking Creation	Create a booking using selected seats and show details.
FR5	Billing	Calculate and display the total booking amount.
FR6	Ticket Generation	Generate ticket information for a confirmed booking.
FR7	Booking Management	Allow the user to view booking or ticket details.
FR8	Admin Management	Allow an administrator to manage show information.
FR9	Validation	Handle invalid choices, unavailable seats and incorrect input safely.

Non-Functional Requirements

- Usability: Console prompts should be simple and understandable.
- Reliability: Seat status and booking totals should remain consistent during a run.
- Maintainability: Responsibilities should be divided among focused classes.
- Error Handling: Invalid input should be handled without unexpected program termination.
- Extensibility: The modular design should allow future GUI or database integration.

Inputs and Outputs

Module	Main Inputs	Expected Outputs
Login	User ID / email, password	Successful login or validation message
Show Selection	Show choice	Selected show details
Seat Booking	Seat numbers	Reserved seats or availability message
Billing	Selected seats / price	Calculated total amount
Ticket	Confirmed booking	Ticket details / booking ID
Admin	Show details and menu choices	Updated show information

System Design

The system follows a modular object-oriented design. Main.java controls the program flow and connects the major modules. User and Customer handle user-related behavior, while Show, Seat, Booking and Ticket represent the core ticket-booking entities. BookingSystem coordinates the overall operations, and Payment-related classes represent the payment step.

Module	Main Responsibility
Main	Controls application flow and console interaction.
User / Customer	Stores user details and customer-specific operations.
Show	Stores show information and show-related operations.
Seat	Represents seat number and availability status.
Booking / Ticket	Stores booking details and generates ticket information.
BookingSystem	Coordinates shows, seats and bookings.
Payment	Represents payment behavior for a booking.
Validation / Exception	Handles invalid input and booking errors.

Use Case Diagram

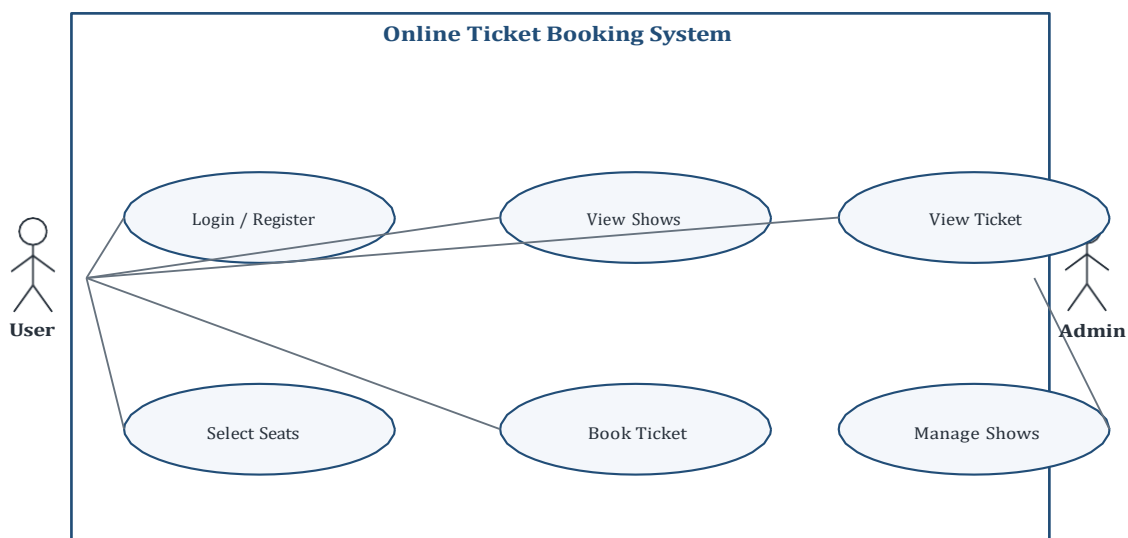


Figure 1. Main interactions of the user and administrator with the Online Ticket Booking System.

Workflow / Process Flow

The booking workflow starts with authentication, followed by show selection and seat selection. The system validates the selected seats before creating the booking. After the total is calculated and the booking is confirmed, ticket information is generated for the user.

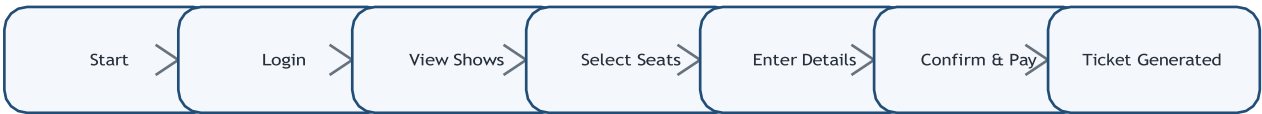


Figure 2. High-level workflow from login to ticket generation.

Sequence Diagram

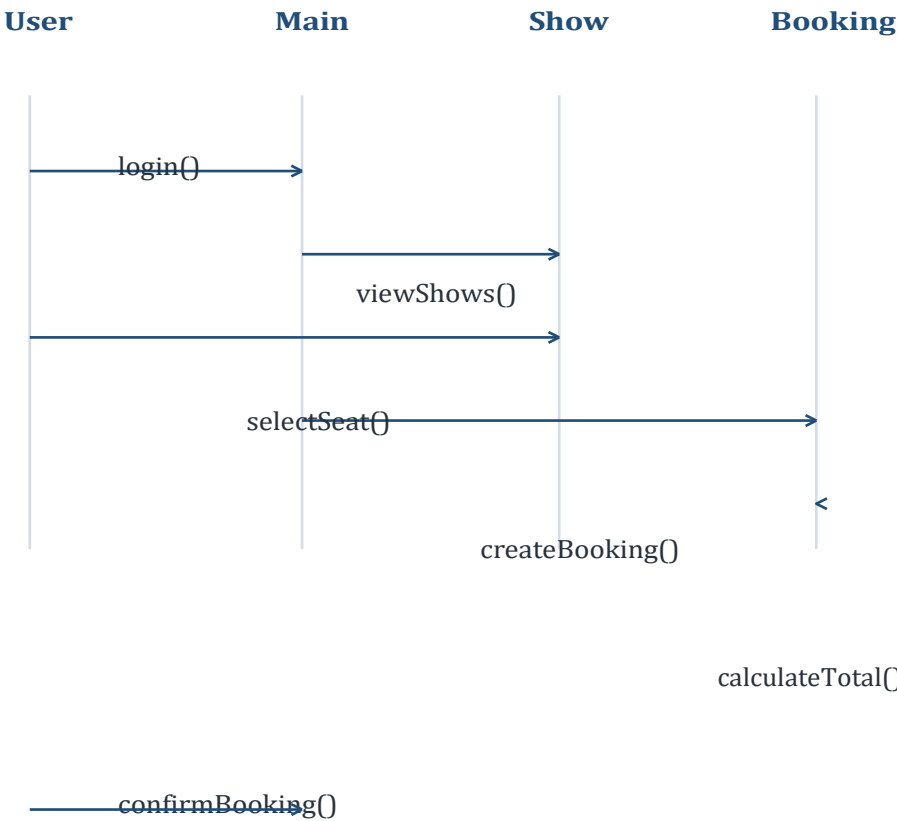


Figure 3. Sequence of interactions during a typical ticket-booking operation.

Class / Component Diagram

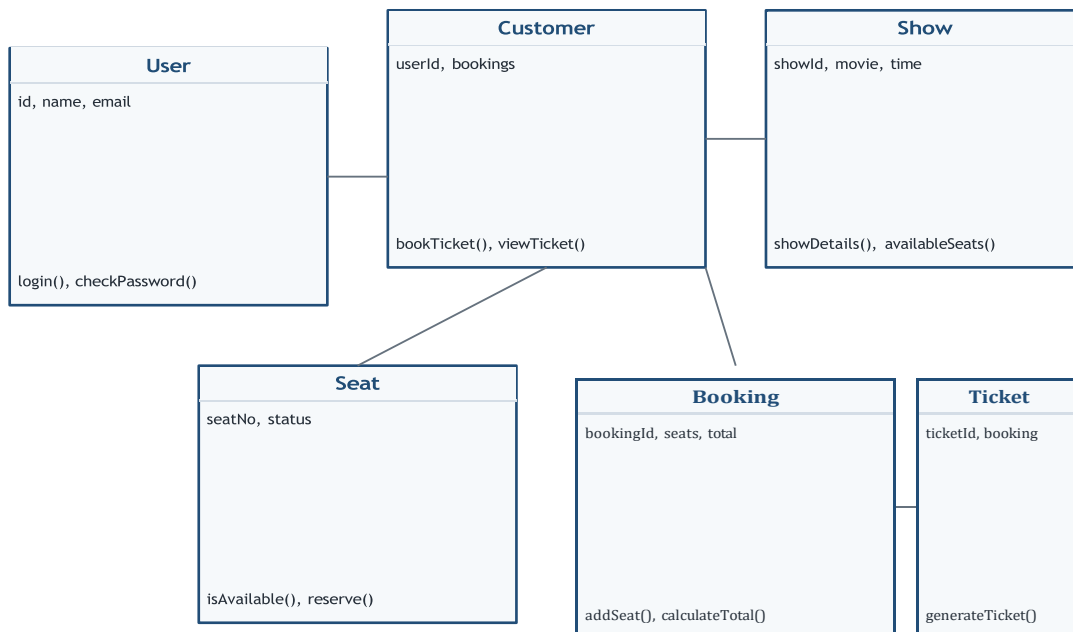


Figure 4. Main Java classes and their relationships.

Design Decisions & Rationale

Decision	Rationale
Modular class design	Each class has a focused responsibility, making the project easier to understand and maintain.
Object-oriented modeling	Users, shows, seats, bookings and tickets are represented as separate objects.
Inheritance / encapsulation	Common user behavior can be reused while sensitive fields remain private.
Collections	Dynamic collections are suitable for maintaining shows, seats and booking items.
Input validation	Prevents invalid menu choices, seat selections and other incorrect values.
Exception handling	Allows booking and input errors to be handled without unexpected termination.
Console interface	Keeps the implementation focused on Java and OOP concepts.

Implementation Details

The application is organized into Java source files under the project source folder. Main.java is used for program flow and user interaction. The remaining classes separate authentication, show management, seat handling, booking, ticket generation and payment-related behavior.

- Main.java — controls the overall application flow and console interaction.
- User.java — stores common user information and login-related behavior.
- Customer.java — implements customer-specific booking operations.
- Show.java — represents an individual show and its details.
- Seat.java — represents a seat and its availability state.
- Booking.java — represents a booking and selected seats.
- Ticket.java — stores ticket information for a confirmed booking.
- BookingSystem.java — coordinates the major booking operations.
- Payment.java / CashPayment.java — represents payment behavior.
- BookingException.java — represents booking-related validation errors.
- FileManager.java — supports file-based data handling where used.

Technologies Used

Technology / Tool	Use in Project
Java	Core programming language.
Object-Oriented Programming	Classes, objects, encapsulation, inheritance and polymorphism.
Java Collections	Dynamic storage for shows, seats and booking information.
Exception Handling	Safe handling of invalid input and booking errors.
Visual Studio Code	Source-code editing and project execution.
Terminal / Command Line	Compilation and execution of the console application.
Git / GitHub	Version control and project submission.

Validation and Error Handling

The system checks user input before using it in booking operations. Typical validation includes checking menu choices, show selection, seat availability and booking-related values. BookingException and related validation logic are used to report errors in a controlled way.

Testing & Results

Testing was performed by executing the main user flows and checking the console output. The tests cover authentication, show access, seat selection, booking creation, billing or total calculation, ticket generation and invalid-input handling

Test Case	Expected Result	Status
Valid user login	User functions become available.	Tested
Invalid login	Access is rejected or an appropriate message is shown.	Tested
View shows	Available shows are displayed.	Tested
Select available seat	Selected seat is accepted.	Tested
Select unavailable seat	Selection is rejected safely.	Tested
Create booking	Selected seats and booking details are stored.	Tested
Calculate total	Correct booking amount is displayed.	Tested
Generate ticket	Ticket details are displayed after confirmation.	Tested
Invalid input	Input is rejected or handled without unexpected termination.	Tested
Admin show management	Administrative menu operation is processed.	Tested

Result

The completed console application supports the intended ticket-booking workflow: users can authenticate, view shows, select available seats, create a booking, calculate the required amount and obtain ticket information. The modular design also provides a separate place for administrative show management and validation logic.

Challenges Faced

- Maintaining consistent communication between multiple Java classes.
- Keeping seat availability synchronized with booking operations.
- Handling incorrect choices and unavailable seats entered through the console.
- Connecting booking creation with total calculation and ticket generation.
- Testing repeated executions with different valid and invalid inputs.

Learnings & Key Takeaways

- Practical understanding of classes, objects and constructors.
- Use of encapsulation and inheritance for organizing related behavior.
- Use of collections for dynamic data storage.
- Importance of modular design and separation of responsibilities.
- Importance of input validation and exception handling.
- Experience compiling and executing a multi-class Java project.

Future Enhancements

- Develop a graphical user interface for customers and administrators.
- Add database storage for users, shows, seats and bookings.
- Add secure online payment integration.
- Provide booking cancellation and refund functionality.
- Generate richer booking and usage reports.
- Add stronger authentication and role-based access control.
- Add automated unit and integration tests.

Conclusion

The Online Ticket Booking System demonstrates how Java and object-oriented programming can be used to model a practical ticket-booking workflow. The modular class structure separates responsibilities for users, shows, seats, bookings, tickets and payment behavior. The resulting console application provides a clear foundation that can later be extended with persistent storage, a graphical interface and additional booking services.

References

- VITyarthi — Build Your Own Project instructions and submission guidelines supplied for the project.
- Java programming language and standard library documentation.
- Project source code developed for the Online Ticket Booking System.

Project Scope & Target Users

The project is intended for a small ticket-booking workflow where users need to browse shows, reserve available seats and receive booking information. The main target users are customers who book tickets and administrators who manage show-related information. The current scope focuses on a console-based Java implementation rather than a production web or mobile platform.

Summary of Major Modules

Module	Purpose
Authentication	Controls access to user functions.
Show Management	Stores and displays show information.
Seat Management	Tracks seat availability and selection.
Booking	Creates bookings and calculates booking details.
Ticket	Produces ticket information after confirmation.
Payment	Represents payment processing behavior.
Validation	Handles incorrect input and booking errors.